Applications of Congruences and Cryptography
Rosen 4.5, 4.6

CS 243 – Discrete Mathematics

sources: K. Rosen, I. Dillig

Congruences

- many practical applications of congruences
  - assign memory locations to computer files
  - generate pseudorandom numbers
  - compute check digits

Hashing Functions

- customer records for companies are identified using a key
  - e.g., social security number
- such keys typically span many more individuals than the company services
  - need a method for using the key to locate a customer
  - need a way to translate the large number to a smaller one
- use a hashing function $h(k)$
  - e.g., $h(k) = k \textbf{ mod } m$
  - m is the number of memory locations
  - should be easy-to-compute and onto

Hashing Functions

- example: find memory locations using the hashing function

  $h(k) = k$ **mod** 111

  for ssn's (a) 064212848 and (b) 037149212

  (a) 064212848 **mod** 111 = 14

  (b) 037149212 **mod** 111 = 65

- hashing functions are not one-to-one, so collisions may occur
  - two keys match the same memory location
  - example: 107405723 **mod** 111 = 14
  - assign to first free memory location afterward

Pseudorandom Numbers

- used for computer simulations
  - linear congruential method:

    $x_{n+1} = (ax_n + c)$ **mod** m

  - for [0..1], divide by m
- example: $a = 7$, $c = 4$, $m = 9$, $x_0 = 3$

  $x_{n+1} = (7x_n + 4)$ **mod** 9

  $x_1 = (21 + 4)$ **mod** 9 = 7

  $x_2 = 8$

  6, 1, 2, 0, 4, 5, 3, 7, 8, …

Check Digits

- add an extra digit at the end of the string for error checking
- parity check bits

  $x_{n+1} = x_1 + x_2 + \ldots + x_n$ **mod** 2

  - 0: even number of 1 bits otherwise, 1
- example: (a) 01100101 (b) 11010110

  (a) $0 + 1 + 1 + 0 + 0 + 1 + 0 \equiv 1$ **mod** 2 assumed no error

  (b) $1 + 1 + 0 + 1 + 0 + 1 + 1 \equiv 1$ **mod** 2 error

Check Digits

- used to check digits for bank account numbers, drivers license numbers, credit card numbers, and UPCs
- UPCs: 12 digits
  - digit 1: product category
  - digits 2-6: manufacturer
  - digits 7-11: product
  - digit 12: check digit determined by

$$3x_1 + x_2 + 3x_3 + x_4 + 3x_5 + x_6 + 3x_7 + x_8 + 3x_9 + x_{10} + 3x_{11} + x_{12} \equiv 0 \textbf{ mod } 10$$

Check Digits

- example:
  - What is the check digit for UPC 79357343104?

    $3 \cdot 7 + 9 + 3 \cdot 3 + 5 + 3 \cdot 7 + 3 + 3 \cdot 4 + 3 + 3 \cdot 1 + 0 + 3 \cdot 4 + x_{12} =$

    $98 + x_{12} \equiv 0 \textbf{ mod } 10$

    so $x_{12} = 2$
  - Is 041331021641 a valid UPC?

    $3 \cdot 0 + 4 + 3 \cdot 1 + 3 + 3 \cdot 3 + 1 + 3 \cdot 0 + 2 + 3 \cdot 1 + 6 + 3 \cdot 4 + 1 = 44$

    $44 \equiv 4 \textbf{ mod } 10$ not valid

ISBNs

- International Standard Book Number (ISBN-10)
  - language, publisher, ID number, check digit (10 = X)

    $x_{10} = \sum_{i=1}^{9} i x_i \pmod{11}$ or

    $\sum_{i=1}^{10} i x_i = 0 \pmod{11}$

  - What is the check digit for ISBN 007288008?

    $x_{10} = 1 \cdot 0 + 2 \cdot 0 + 3 \cdot 7 + 4 \cdot 2 + 5 \cdot 8 + 6 \cdot 8 + 7 \cdot 0 + 8 \cdot 0 + 9 \cdot 8$

    $x_{10} \equiv 189 \textbf{ mod } 11 \equiv 2 \textbf{ mod } 11$

    $x_{10} = 2$

  - Is 084930149X a valid ISBN-10?

    $1 \cdot 0 + 2 \cdot 8 + 3 \cdot 4 + 4 \cdot 9 + 5 \cdot 3 + 6 \cdot 0 + 7 \cdot 1 + 8 \cdot 4 + 9 \cdot 9 + 10 \cdot 10 = 299$

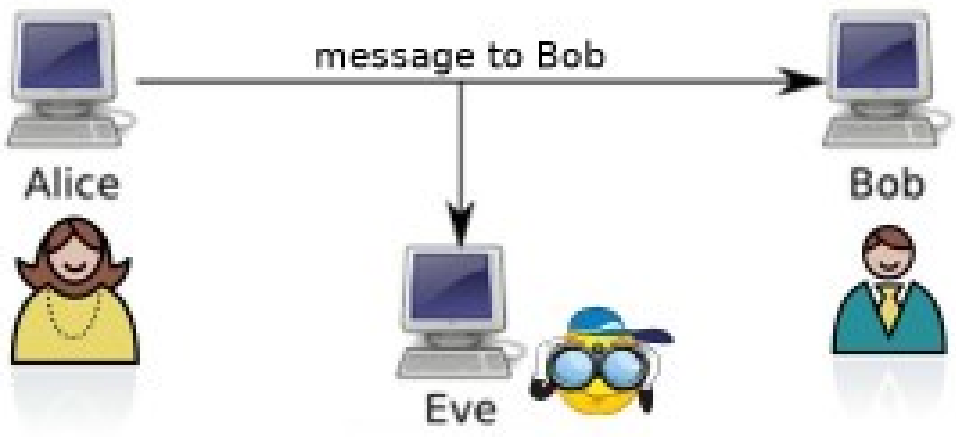
    $299 \textbf{ mod } 11 \equiv 2 \textbf{ mod } 11$ not valid since it should be $0 \textbf{ mod } 11$

ISBNs

- ISBN-13 introduced to handle more books
- can detect
  - single errors
  - transposition errors

Cryptography

- cryptography is the study of techniques for safely transmitting information in the presence of adversaries



- how can Alice and Bob communicate securely without Eve being able to read their messages?
- number theory plays a key role

Cryptography Systems

- two types of cryptography systems
  - private (secret) key
    - sender and receiver agree on secret key that both use to encrypt/decrypt messages
  - public key
    - a public key is used to encrypt the message, and a private key is used to decrypt the message

Private Key Cryptography

- classical method used for thousands of years
- Julius Caesar used a shift cipher where each letter in the alphabet is shifted a certain number of locations (with end-around looping)

$$f(p) = (p + 3) \textbf{ mod } 26 \qquad p \leq 25$$

$$\mathbf{Z}_{26} = 0..25$$

- Analogy: Alice wants to send Bob briefcase with secret message; they have a common key to lock/unlock briefcase
  - Alice locks briefcase with shared key and Bob unlocks briefcase with shared key
  - only works well when number of parties involved in communication is small

Private Key Cryptography

- Julius Caesar shift cipher

$$f(p) = (p + 3) \textbf{ mod } 26 \qquad p \leq 25$$

  - example: what is the secret message produced from the message "MEET YOU TOMORROW"
    - assign each letter its number from $\mathbf{Z}_{26}$

      12 4 4 19 24 14 20 8 13 19 7 4 15 0 17 10

    - replace with $f(p) = (p + 3)$ **mod** 26

      15 7 7 22 1 17 23 11 16 22 10 7 18 3 20 13

      PHHW BRX LQ WKH SDUN

Private Key Cryptography

- to recover the message, use

$$f^{-1}(p) = (p - k) \textbf{ mod } 26 \qquad p \leq 25 \text{ and } k \text{ is the key}$$

  - example: decrypt "OLSSV DVYSK"
    - assign each letter its number from $\mathbf{Z}_{26}$

      14 11 18 18 21 3 21 24 18 10

    - replace with $f^{-1}(p) = (p - 7) \textbf{ mod } 26$

      7 4 11 11 14 22 14 17 11 3

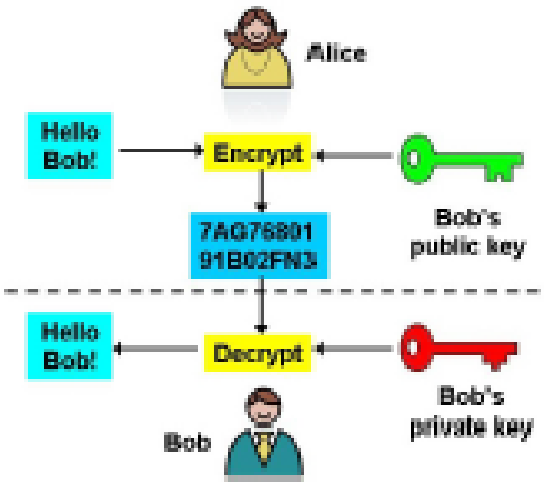
      HELLO WORLD

Public Key Cryptography

- modern method since the 1970s
- different keys used to encrypt and decrypt message
- how can parties exchange information using different keys?
  - Analogy: Alice puts message in briefcase, locks with her own key A, sends to Bob
  - Bob gets locked briefcase, adds his lock B, sends back to Alice
  - Alice gets double locked box, removes A, sends back to Bob
  - Bob opens briefcase using his own key
- no key exchange required
  - modern methods, such as RSA, work this way

RSA Cryptography

- named after researchers Rivest, Shamir, and Adleman
- Analogy:



- Bob has two keys: public and private
- Everyone knows Bob's public key, but only he knows his private key
- Alice encrypts message using Bob's public key
- Bob decrypts message using his private key
- public key can encrypt, but not decrypt; therefore, no one can read message except Bob

RSA Math

- in RSA, the private key consists of two very large prime numbers p and q
- the public key consists of a number n, which is the product of p, q and another number e
  - e is a number relatively prime with $(p - 1)(q - 1)$
- encrypt messages using n and e, but to decrypt, must know p and q
- in theory, p and q can be extracted from n using prime factorization, but this is intractable for very large numbers (computational time of billions of years)
  - security of RSA relies on inherent computational difficulty of prime factorization

RSA Encryption

- to send a message to Bob, Alice first represents the message as a sequence of numbers, M
- Alice then uses Bob's public key information, n and e, to perform encryption

$$C = M^e \ (\mathbf{mod}\ n)$$

- C is called the cipher text

RSA Encryption Example

- Encrypt the message ”STOP” using RSA with n = 2537, e = 13
  - first, convert each letter to a number in [0, 25]:

    S = 18,T = 19,O = 14,P = 15
  - group sequence into blocks of 4 digits:

    M = 1819 1415
  - encrypt each block as $C = M^{13}$ (**mod** 2537)

    1819^{13} (**mod** 2537) = 2081

    1415^{13} (**mod** 2537) = 2182
  - cipher text: 2081 2182

RSA Decryption

- decrypt cipher text using private keys p, q
  - decryption key d is the inverse of e modulo $(p - 1)(q - 1)$:

    $$d \cdot e \equiv 1 \ (\textbf{mod}\ (p - 1)(q - 1))$$

  - d can be computed reasonably efficiently if we know $(p - 1)(q - 1)$
    - others who do not know p and q cannot compute d in a reasonable amount of time

RSA Decryption

- using the Chinese remainder theorem and Fermat's Little Theorem, it can be shown that:

$$(M^e)^d \equiv M \ (\mathbf{mod}\ n)$$

- since the ciphertext C is just M^e, C^d (**mod** n) allows for decrypting the message
- since Bob can compute d using p and q, Bob can easily decrypt the message, but no one else can

RSA Decryption Example

- Decrypt the cipher text 0981 0461 using RSA with $p = 43$, $q = 59$, and $e = 13$
  - first, compute d, the inverse of e modulo $(p - 1)(q - 1)$

    $$13x \equiv 1 \ (\textbf{mod}\ (43 - 1)(59 - 1))$$

    $$13x \equiv 1 \ (\textbf{mod}\ 2436)$$

  - compute s and t such that

    $$13s + 2436t = 1$$

  - using Euclidean algorithm, $s = 937$, $t = -5$

RSA Decryption Example

- Decrypt the cipher text 0981 0461 using RSA with p = 43, q = 59, and e = 13 (continued)
  - solution to this system is given by

    $$x = \frac{sb}{t} + \frac{m}{d}u \quad \text{where } u \in \mathbf{Z}$$

    s = 937, b, d = 1, and m =2436; therefore, $\bar{e}$ = 937

    0981^{937} (**mod** 2537) = 0704

    0461^{937} (**mod** 2537) = 1115
  - decrypted message 07 04 11 15 = “HELP”

RSA Security

- the encryption function used in RSA is a trapdoor function
  - easy to compute in one direction, but very difficult in the reverse direction without additional knowledge
- encryption is easy since it only requires exponentiation and mod
- decryption without the private key is very hard because it requires prime factorization
  - security of RSA depends on difficulty of prime factorization

RSA Security

- decryption without the private key is very hard because it requires prime factorization
  - security of RSA depends on difficulty of prime factorization
  - as computers become more powerful and factorization algorithms better, larger integers can possibly be factored
  - over time, we must use larger and larger prime numbers to ensure secure communication
    - today, RSA is secure using prime number with more than 200 digits